Algorithms for Data Science

Lecture #3 #4: Analyzing Algorithms

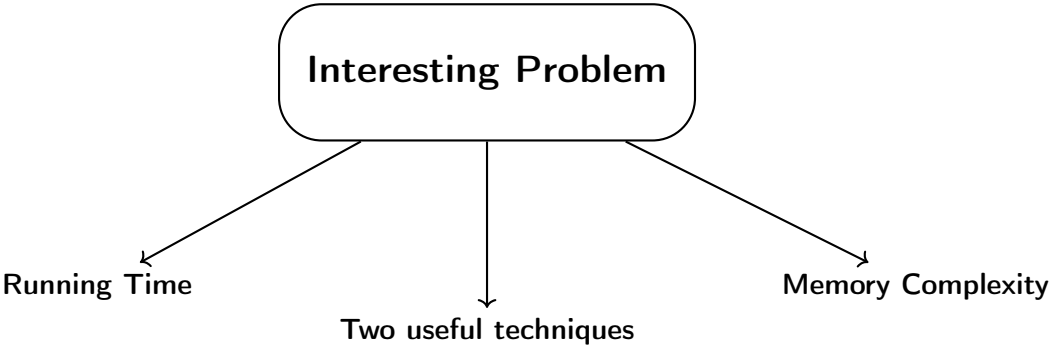
Mateusz Buczyński

University of Warsaw

04, 11.03.2026

Thanks to dr Krzysztof Fleszar for the base material.

This Lecture



Recap: Running Time vs Memory Complexity

Running Time

- Number of basic operations.
- Examples: $+$, $-$, $/$, $\times$, $=$, $\leq$, $\ldots$

Memory Complexity

- Number of memory cells used (RAM model).
- Intuition: how many values we store at the same time.

Common perspective

Both are studied as functions of the input size n (or another key parameter).

Recap: Example

Example

Create an array of size n filled with 0s.

Reasoning

- Allocate n cells.
- Write 0 into each cell once.

Recap: Example

Example

Create an array of size n filled with 0s.

Reasoning

- Allocate n cells.
- Write 0 into each cell once.

Complexities

- Time: $\Theta(n)$
- Space: $\Theta(n)$

"Oh"-Notation: Picking Useful Bounds

For a running-time function $f(n)$

- $f(n) = O(g(n))$: g is an asymptotic upper bound
- $f(n) = \Omega(g(n))$: g is an asymptotic lower bound
- $f(n) = \Theta(g(n))$: asymptotically tight (both upper and lower)

Example: $f(n) = 10n + n^2$

$10n+n^2 = \Omega(1)$, $10n+n^2 = O(1000n^{1000})$

These are correct, but not very helpful.

Our goal

Find bounds that match as closely as possible:

$\Omega(\cdot)$ and $O(\cdot)$ meet at $\Theta(\cdot)$.

"Oh"-notation - example

Sandwich the function

Let $f(n) = n^2 + 10n$.
For sufficiently large n:

$$n^2 \leq 10n + n^2 \leq 10n^2 + n^2 = 11n^2$$

$10n + n^2 = \Omega(n^2)$ $\qquad$ $10n + n^2 = O(n^2)$

$$\Rightarrow \; 10n + n^2 = \Theta(n^2)$$

Interpretation

$O(\cdot)$ is commonly used for worst-case upper bounds on running time, $\Omega(\cdot)$ gives lower bounds, and $\Theta(\cdot)$ gives the tight growth rate.

Asymptotic Notation Is Not Symmetric

Unlike ordinary equality

Ordinary equality is bidirectional:

$$f(n) = g(n) \iff g(n) = f(n).$$

But asymptotic statements are directional:

$$f(n) = O(g(n)) \implies g(n) = O(f(n)).$$

Counterexample

$$n = O(n^2) \quad \text{but} \quad n^2 \neq O(n).$$

Simple Analysis: Constant-Size Input

Algorithm

Input: k, m

$$\texttt{tmp} = k + m$$

$$\texttt{tmp} = \texttt{tmp}/2$$

return tmp

Complexity intuition

- Dominating operation count is constant (about 1 key arithmetic step).
- Uses one extra variable (tmp).

Simple Analysis: Constant-Size Input

Algorithm

Input: k, m

$$\texttt{tmp} = k + m$$

$$\texttt{tmp} = \texttt{tmp}/2$$

return tmp

Complexity intuition

- Dominating operation count is constant (about 1 key arithmetic step).
- Uses one extra variable (tmp).

$$\text{Time} = \Theta(1),$$

Simple Analysis: Constant-Size Input

Algorithm

Input: k, m

$$\texttt{tmp} = k + m$$

$$\texttt{tmp} = \texttt{tmp}/2$$

return tmp

Complexity intuition

- Dominating operation count is constant (about 1 key arithmetic step).
- Uses one extra variable (tmp).

$$\text{Time} = \Theta(1), \qquad \text{Extra Space} = \Theta(1)$$

Simple Analysis: Input Size n (Two Patterns)

A) Sum first half

Input: $a[1..n]$ (assume n even)

```
r = 0
for i = 1..n/2:
   r += a[i]
return r
```

Simple Analysis: Input Size n (Two Patterns)

A) Sum first half

Input: $a[1..n]$ (assume n even)

```
r = 0
for i = 1..n/2:
   r += a[i]
return r
```

Cost

- About $n/2$ loop updates $\Rightarrow \Theta(n)$ time.
- Extra variables: r and $i \Rightarrow \Theta(1)$ space.

B) Prefix-sum array

Input: $a[1..n]$

```
b[1..n] initialized with 0
for i = 1..n:
   b[i] = a[i]
for i = 2..n:
   b[i] += b[i-1]
return b[n]
```

Simple Analysis: Input Size n (Two Patterns)

A) Sum first half

Input: $a[1..n]$ (assume n even)

```
r = 0
for i = 1..n/2:
   r += a[i]
return r
```

Cost

- About $n/2$ loop updates $\Rightarrow \Theta(n)$ time.
- Extra variables: r and $i \Rightarrow \Theta(1)$ space.

B) Prefix-sum array

Input: $a[1..n]$

```
b[1..n] initialized with 0
for i = 1..n:
   b[i] = a[i]
for i = 2..n:
   b[i] += b[i-1]
return b[n]
```

Cost

- Dominating operations: $n + (n-1)$ (plus initialization) $\Rightarrow \Theta(n)$ time.
- Stores array $b[1..n]$ (and loop index) $\Rightarrow \Theta(n)$ space.

More Parameters

Input with two independent sizes

$$a[0..k-1], \quad b[0..m-1], \quad n = k + m$$

Key idea

When an algorithm depends on both k and m, keeping both parameters can give a more informative comparison than using only n.

Example: Two Similar Worst Cases, Different Behavior

Algorithm A

```
[i for i in range(m*m + k)]
```

$$T_A(k, m) = \Theta(m^2 + k)$$

Algorithm B

```
[i for i in range(m + k*k)]
```

$$T_B(k, m) = \Theta(m + k^2)$$

Worst-case in terms of n

- T_A: if $k = 0,\ m = n$, then $T_A = \Theta(n^2)$.
- T_B: if $m = 0,\ k = n$, then $T_B = \Theta(n^2)$.

Teaching takeaway

Both are $\Theta(n^2)$ in the worst case, but $\Theta(m^2 + k)$ vs. $\Theta(m + k^2)$ enables a better comparison for specific k, m regimes.

Main Example Problem: Eye Tracking

Setup

We model a screen (width W, height H) with:

- n gaze points (where users looked),
- k axis-aligned rectangles (regions of interest, e.g., UI elements or ads).

Query / Output

For each rectangle, report how many gaze points fall inside it.

Given k rectangles, compute k counts.

Interpretation

Each rectangle count estimates user attention on that region.

Why This Problem Matters

Applications

- Website design
- User interfaces
- Cockpit design

Applications

- Ad placement
- Gaming

Algorithmic View

This is a geometric counting problem:
points + rectangles $\rightarrow$ count points per rectangle.

Simplified Problem: From 2D to 1D

2D version

- Points in the plane
- Axis-aligned rectangles
- Query: how many points are inside each rectangle?

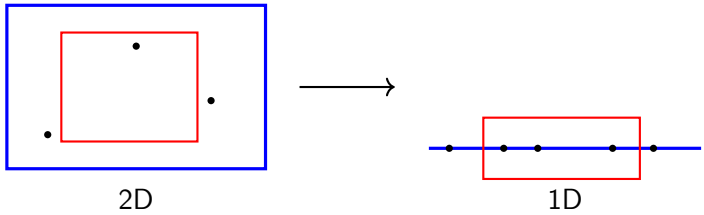
1D version

- Points on a line
- Intervals
- Query: how many points are inside each interval?

Key idea

We simplify geometry: **rectangles** become **intervals**, while the counting goal stays the same.

Simplified Problem



Simplified Problem: Why This Helps

Decision rule

First solve the 1D counting problem cleanly, then reuse the same analysis pattern for 2D.

What is preserved?

- Same type of output: one count per query object
- Same algorithmic question: fast repeated range counting
- Same evaluation criteria: running time and memory

Simplified Problem

Input

- n points on a line segment in $[0, W)$
- k intervals, each with integer endpoints

Output

Compute an array $\texttt{res}[1..k]$ such that

$$\texttt{res}[i] = \text{number of points in the } i\text{-th interval.}$$

Notation reminder

If I denotes the i-th interval, then $\texttt{res}[I]$ is just shorthand for $\texttt{res}[i]$.

Example: $n = 8,\ k = 2$

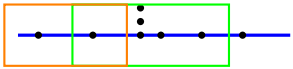
Intervals

- l_1: first (left) interval
- l_2: second (right) interval

Counts

$$\texttt{res}[1] = 2$$

$$\texttt{res}[2] = 6$$



Simplified Problem: Baseline Idea

Input

- n points on a line
- k intervals
- Integer coordinates in $[0, W)$

Output

For each interval I:

$$\texttt{res}[I] = \#\{\text{points inside } I\}.$$

Solution 1 (naïve)

```
res[1..k] = 0
for each interval I:
   for each point p:
      if p in I:
         res[I]++
```

- k intervals $\times$ n points
- Membership test in $\Theta(1)$

Time: $\Theta(nk)$

Extra space: $\Theta(k)$

Simplified Problem: Counting Approach

Key observation

Coordinates are integers in $[0, W)$, so we can count points per coordinate.

Step 1: Build counts

```
counter[0..W-1] = 0
for each point p:
   x = coordinate of p
   counter[x]++
```

Time: $\Theta(n)$, Space: $\Theta(W)$

Step 2: Answer intervals

```
res[1..k] = 0
for each interval I:
   for each x in I:
      res[I] += counter[x]
```

Time: $\Theta(kW)$ (worst case), Space: $\Theta(k)$

$$\Theta(W + n + k + kW) = \Theta(n + kW)$$

Result

Total Time: $\Theta(n + kW)$

Simplified Problem: Cumulative Sums

Setup

We have:

- n points on integer coordinates $0, 1, \ldots, W$
- k interval queries

Let $c[x]$ be the number of points at coordinate x.

Prefix idea

Define

$$P[x] = \sum_{i=0}^{x-1} c[i]$$

So $P[x]$ is the number of points strictly to the left of coordinate x.

Why this helps

Once P is built, each query is answered by one subtraction.

Answering an Interval Query

Decision rule

For interval between coordinates ℓ and r:

$$\#(\text{points in } [\ell, r)) = P[r] - P[\ell]$$

If you want closed intervals

$$\#(\text{points in } [\ell, r]) = P[r+1] - P[\ell]$$

(assuming integer coordinates).

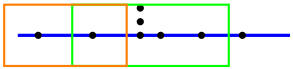
From the picture

If

$$P[\ell] = 1, \quad P[r] = 7$$

then

$$\#\text{ in between} = 7 - 1 = 6.$$



Cumulative Sums Algorithm

Step 1: Count points per coordinate

$$\texttt{counter}[0..W-1] \leftarrow 0$$

For each point p at coordinate x_p:

$$\texttt{counter}[x_p] \leftarrow \texttt{counter}[x_p] + 1$$

Step 2: Build prefix sums

$$\texttt{cumsum}[0] \leftarrow 0$$

For $x = 1, \ldots, W$:

$$\texttt{cumsum}[x] = \texttt{cumsum}[x-1] + \texttt{counter}[x-1]$$

So $\texttt{cumsum}[x]$ = number of points strictly left of x.

Complexity

$$\Theta(n)+\Theta(W)+\Theta(k) = \Theta(n+k+W)$$

Space

$\Theta(W)$ for $\texttt{counter}$, $\texttt{cumsum}$

Comparison

- Sol 1: $\Theta(n \cdot k)$
- Sol 2: $\Theta(n + k \cdot W)$
- Sol 3: $\Theta(n + k + W)$

Simplified Problem: Formal Statement

Input

Given three integer arrays:

$$x \in \{0, \ldots, W-1\}^n, \qquad x_0, x_1 \in \{0, \ldots, W-1\}^k$$

Equivalently, for every value v appearing in x, x_0, x_1:

$$0 \leq v < W.$$

Output

Compute an integer array res of size k such that

$$\mathrm{res}[i] = |\{\, j \mid x_0[i] \leq x[j] \leq x_1[i] \,\}| \quad \text{for } i = 0, \ldots, k-1.$$

What Does $\mathrm{res}[i]$ Count?

i-th query interval

$$[x_0[i],\ x_1[i]]$$

- The pair $(x_0[i], x_1[i])$ defines the i-th interval.
- $\mathrm{res}[i]$ is the number of positions j in array x whose value lies in that interval.
- Bounds are inclusive: $x_0[i] \leq x[j] \leq x_1[i]$.

Simplified Problem: Python

Compact implementation

```
counts = [0 for _ in range(W)]
for i in range(n):
    counts[x[i]] += 1

cumsum = [0 for _ in range(W + 1)]
for p in range(1, W + 1):
    cumsum[p] = cumsum[p - 1] + counts[p - 1]

res = [0 for _ in range(k)]
for j in range(k):
    res[j] = cumsum[x1[j] + 1] - cumsum[x0[j]]
```

Eyetracking Problem: Recall Main 2D Problem

Input

- Screen size: $W \times H$
- n gaze points in the plane
- k axis-aligned query rectangles

Output

For each rectangle i, report:

$$\text{res}[i] = \#\{\text{points inside rectangle } i\}.$$

- Same dataset of points, many rectangle queries.
- Goal: fast counting per rectangle.

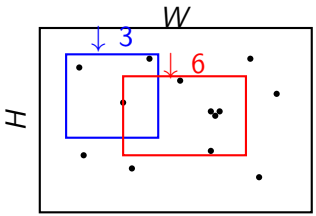
Visual Intuition: Count Points in Each Rectangle

Example query answers

- Blue rectangle contains 3 points.
- Red rectangle contains 6 points.

What changes across queries?

- Points are fixed.
- Rectangle boundaries change.
- We repeat counting k times.



Solution 1: Brute-Force Counting (2D)

Input

- Points: arrays x, y of size n, where point j is $(x[j], y[j])$.
- Rectangles: arrays x_0, y_0, x_1, y_1 of size k, where rectangle i has opposite corners

$$(x_0[i], y_0[i]) \quad \text{and} \quad (x_1[i], y_1[i]).$$

Output

Compute res$[0..k-1]$, where

$$\text{res}[i] = |\{j : \ x_0[i] \le x[j] \le x_1[i] \ \wedge \ y_0[i] \le y[j] \le y_1[i]\}| \, .$$

- Borders are included ($\le$, not $<$).
- Same points are reused for all k rectangle queries.

Solution 1: Direct Double Loop

Reference implementation (C++ style)

```
vector<int> res(k, 0);

for (int i = 0; i < k; i++) {
  for (int j = 0; j < n; j++) {
    if (x0[i] <= x[j] && x[j] <= x1[i] &&
        y0[i] <= y[j] && y[j] <= y1[i]) {
      res[i]++;
    }
  }
}
```

Time

$\Theta(k \cdot n)$

Memory

$\Theta(k)$ for output res

($\Theta(1)$ extra beyond output).

Analyzing Solution 1: Running Time

What the algorithm does

- Create `res` with k entries initialized to 0.
- For each rectangle index $i = 0, \ldots, k-1$, scan all points $j = 0, \ldots, n-1$.
- For each pair (i, j), check:

$$x_0[i] \le x[j] \le x_1[i] \quad \wedge \quad y_0[i] \le y[j] \le y_1[i]$$

- If true, increment $\texttt{res}[i]$.

Operation count

Each inner check is constant time: $\Theta(1)$.
Total checks: $k \cdot n$.

$$T(n, k) = k \cdot n \cdot \Theta(1) = \Theta(nk)$$

Analyzing Solution 1: Memory Complexity

Core idea

- The algorithm allocates an output array `res` of size k: `vector<int> res(k, 0)`.
- Two loop counters (i, j) are the only temporary variables.
- Input arrays are provided to the algorithm (not newly allocated inside the loops).

Space contributions

$$S_{\text{input}} = (n, k, w, h) + (x, y) + (x_0, x_1, y_0, y_1) = 4 + 2n + 4k = \Theta(n + k)$$

$$S_{\text{aux}} = \Theta(1) \qquad S_{\text{output}} = \Theta(k)$$

What Should We Report?

Auxiliary space

$$S_{\text{aux}} = \Theta(1)$$

- Loop indices i, j
- No other growing temporary structure

Output space

$$S_{\text{output}} = \Theta(k)$$

- Array `res` stores one answer per rectangle

Final complexity statements

- If reporting *extra space including output*: $\Theta(k)$
- If reporting *auxiliary space only*: $\Theta(1)$
- If including input storage: $\Theta(n + k)$

Solution 2: Counting Approach

Initialization

```
vector<vector<int>> counts(H, vector<int>(W, 0));
```

How to read this line

- `counts` is a vector of size H (so: H rows).
- Each row is an `int` vector of size W (so: W columns).
- Every entry starts as 0.

Solution 2: Counting Table Intuition

Structure

$$\texttt{counts} = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}$$

H rows
W columns

Meaning of one cell

`counts[b][a]`

stores the number of points located at pixel (a, b).

Key idea

Count once into `counts`, then answer rectangle queries from table sums.

Counting approach: answer each rectangle query

For rectangle i with corners $(x_0[i], y_0[i])$ to $(x_1[i], y_1[i])$

$$\texttt{sum} \leftarrow 0$$

$$\text{for } b = y_0[i] \text{ to } y_1[i]: \quad \text{for } a = x_0[i] \text{ to } x_1[i]: \quad \texttt{sum} \mathrel{+}= \texttt{counts}[b][a]$$

Then append the answer for this rectangle:

```
res.push_back(sum)
```

What this is doing

For each query rectangle, scan all coordinates inside it and add the number of points stored in each cell.

Solution 2: Counting Approach

Pseudo-code

```
vector<vector<int>> counts(H, vector<int>(W, 0));
for (int i = 0; i < n; i++) counts[y[i]][x[i]]++;

vector<int> res;
for (int i = 0; i < k; i++) {
    int sum = 0;
    for (int b = y0[i]; b <= y1[i]; b++)
        for (int a = x0[i]; a <= x1[i]; a++)
            sum += counts[b][a];
    res.push_back(sum);
}
```

- 'counts[b][a]' = number of points at coordinate (a, b).
- 'res[i]' = number of points inside rectangle i.

Solution 2: Running Time

Preprocessing

- Build ‘counts‘ array: $\Theta(HW)$
- Insert n points into ‘counts‘: $\Theta(n)$

Per rectangle

- Initialize ‘sum‘: $\Theta(1)$
- Scan all cells in rectangle:

$$\Theta\big((x_1 - x_0 + 1)(y_1 - y_0 + 1)\big)$$

Worst case

A rectangle can cover almost the full grid:

$$\text{width} = \Theta(W), \quad \text{height} = \Theta(H)$$

So one query costs $\Theta(WH)$

For k rectangles

$$\Theta(kWH)$$

Total

$$\Theta(n + kWH)$$

Two Baseline Approaches

Solution 1: Direct counting

For each rectangle, scan all points and count those inside.

$$\Theta(n \cdot k)$$

Solution 2: Grid counting approach

- Place points into a $W \times H$ counting grid
- For each rectangle, sum covered grid cells

$$\Theta(n + k \cdot W \cdot H)$$

Takeaway

Both are correct baselines; the second shifts work to the grid and can help when many queries are asked on the same screen.

Solution 3: Cumulative Sums

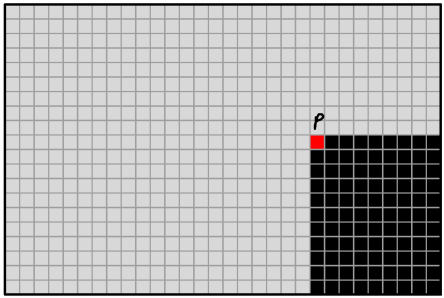
Key idea

Precompute a table so each cell stores the number of points in a large suffix region:

$$S[x, y] = \#\{\text{points in } [x, W) \times [y, H)\}.$$

How to read the picture

- Pick one cell (the red cell).
- Count all points in the black rectangle starting at that cell and extending to the bottom-right corner.
- Save that count as $S[x, y]$.



Solution 3: Cumulative Sums

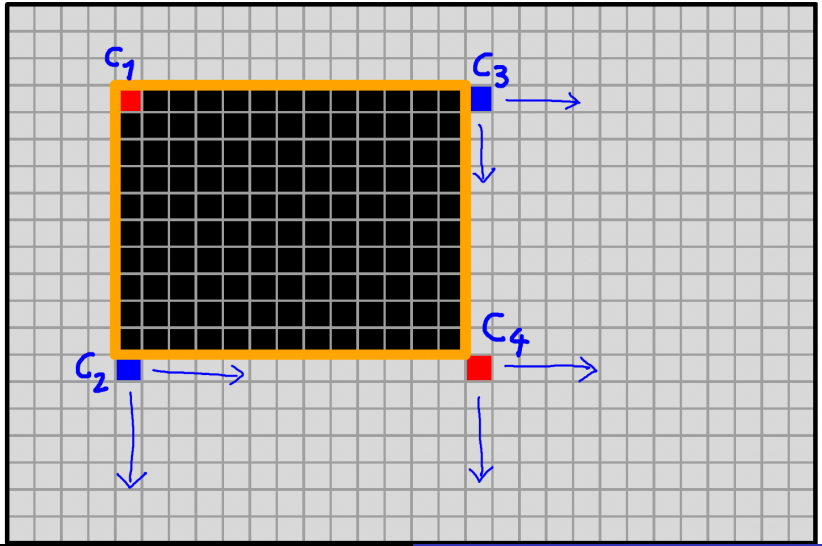
Inclusion–exclusion with four corners

Let c_1 (top-left), c_2 (bottom-left), c_3 (top-right), and c_4 (bottom-right) be the aligned reference corners. Then

$$\#(\text{points in rectangle}) = \mathrm{cumsum}[c_1] - \mathrm{cumsum}[c_2] - \mathrm{cumsum}[c_3] + \mathrm{cumsum}[c_4].$$

- $\mathrm{cumsum}[c_1]$ starts from a larger suffix region that contains the target rectangle.
- Subtracting $\mathrm{cumsum}[c_2]$ and $\mathrm{cumsum}[c_3]$ removes the outside bottom and right parts.
- Their intersection (bottom-right overlap) is subtracted twice, so add back $\mathrm{cumsum}[c_4]$ once.

Solution 3: Cumulative Sums



Solution 3: Query Cost

Per rectangle

The formula uses a constant number of table lookups and arithmetic operations.

$$T_{\text{one query}} = \Theta(1)$$

For k rectangles

$$T_{\text{all queries}} = \Theta(k)$$

Important caveat

This query cost holds **assuming** cumsum **is already precomputed!**.

Solution 3: Cumulative Sums

How do we compute cumulative sums?

We first build a grid that tells us how many points fall on each pixel.

Step 1: Count points on each pixel

For each pixel (a, b), define

$$\texttt{counts}[b][a] \;=\; \#\{\text{points with coordinates } (a, b)\}.$$

Interpretation

- Most pixels have value 0.
- A pixel with one point has value 1.
- If multiple points share a pixel, the value is greater than 1.

Solution 3: Cumulative Sums

Two arrays to build

- **Step 1:** Compute the number of points *on each pixel*:

$$P(x, y) = \#\{\text{points at pixel } (x, y)\}$$

- **Step 2:** Compute the number of points *to the right of each pixel*:

$$R(x, y) = \#\{\text{points strictly right of } (x, y)\}$$

Solution 3: Cumulative Sums

Three counting steps

1. Count points on each pixel (x, y).
2. For each pixel, count points to the right on the same row.
3. For each pixel, count points in the full "black area" (suffix rectangle).

Cost of cumsum table

$$\Theta(n + W \cdot H)$$

- Point insertion: $\Theta(n)$
- DP over grid: $\Theta(W \cdot H)$

Key observation

Each pixel update is $\Theta(1)$, and there are $W \cdot H$ pixels.

Solution 3: Final Running Time

Query phase

After preprocessing, each rectangle answer uses a constant number of table lookups:

$$\Theta(1) \text{ per query} \quad \Rightarrow \quad \Theta(k) \text{ for all queries.}$$

Total complexity of Solution 3

$$\Theta(n + W \cdot H) + \Theta(k) = \Theta(n + k + W \cdot H).$$

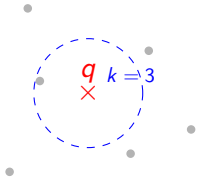
Comparison

Sol 1: $\Theta(nk)$, Sol 2: $\Theta(n + kW \cdot H)$, Sol 3: $\Theta(n + k + W \cdot H)$.

kNN Search Problem

Input

- Dataset $X = \{x_1, \ldots, x_n\}$ in d dimensions
- Query point $q \in \mathbb{R}^d$
- Parameter k (number of neighbors)



Output

The k points from X with the smallest distance to q.

$$\text{dist}(q, x_i) = \|q - x_i\|_2$$

Algorithmic Goal

How do we find these neighbors efficiently as n and d grow?

Stage 1: Brute Force kNN

Algorithm

```
best = [] # max-heap of size k
for i in range(n):
    d_i = compute_dist(q, X[i]) # O(d)
    update_heap(best, d_i, i)   # O(log k)
return best
```

Complexity

- **Time:** $\Theta(n(d + \log k))$
- **Extra Space:** $\Theta(k)$

Performance

Works for any d, but is linear in n.
Too slow for massive datasets.

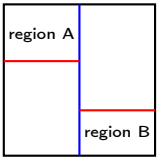
Stage 2: Exact Search (KD-Trees)

Preprocessing: $\Theta(n \log n)$

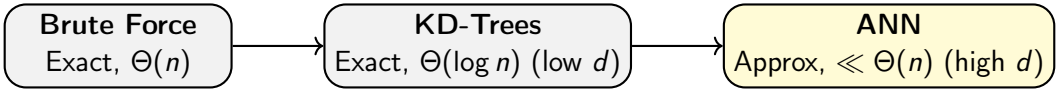
Recursively split space into regions using axis-aligned medians.

Query: $\approx \Theta(\log n + k)$

- Use the tree to prune branches that are too far.
- **High d issue:** "Curse of dimensionality" makes pruning fail ($\Theta(n)$).



Stage 3: Approximate kNN (ANN)



The Trade-off

- We sacrifice Recall (finding the 100% true neighbors).
- We gain Speed (sublinear query time).

Techniques

- **LSH:** Random projections.
- **Graphs (HNSW):** Navigating small-world networks.

Takeaway

In Data Science, we almost always use **Stage 3** for production scale.


ringier axel springer TECH SUMMER
A może tego lata zrobisz coś dla milionów użytkowników? Aplikuj na staż!
Aplikuj do 29 marca!
Ścieżki: Data Science, UX, Project Management, Quality, AI Fullstack, Backend, Mobile oraz Fullstack Internship.
techsummer.ringieraxelspringer.com